



Part 3: Compile Time vs Runtime & Variables vs Constants

Simple Study Notes for Beginners

1. Compile Time vs. Runtime

Every line of code goes through two steps before it opens on a phone:

⚡ Compile Time ("Before the app runs")

- The computer scans your code layout to see how you built it.
- It looks for simple grammar mistakes (like a missing semicolon ; or a misspelled word).
- It checks for mismatched data types (like trying to put text inside a whole number box).
- **If there is a mistake:** The app **never starts**, and you see a red error right away.
- **One-Liner Summary:** Convert your code into binary.

🕒 Runtime ("While the app is running")

- The computer executes your code instructions line-by-line.
- The app is active, alive, and running on a device.
- **If there is a mistake:** The app starts up fine, but it suddenly **crashes in the middle** later on (like when you tap a broken button).
- **One-Liner Summary:** Execute that binary code.

💡 Plain English Analogy

- **Compile Time:** The teacher checks your homework sheet layout before submission. If you spelled words wrong, you can't hand it in.
- **Runtime:** You handed in your homework, and the teacher is reading it. Halfway down, she finds a mistake and fails the paper.

Phase	When?	Error Example	What happens?	Dart/Flutter Shortcut
⚡ Compile Time	Before you run.	<code>int x = "hello";</code>	Program never starts.	You see the error before the app opens on your phone.
		<code>print(list[999]);</code>		

Phase	When?	Error Example	What happens?	Dart/Flutter Shortcut
 Runtime	While it's running.		Program starts, then crashes.	The app opens, and then crashes later when you tap a button.

2. Variables vs. Constants

A **data type** tells the computer what kind of data you are storing in a box (like text or numbers), how much memory space to save, and what tasks you are allowed to do with it. You can check any item's type with `variable.runtimeType`.

Variables (Can Change)

A variable uses code that can be rewritten with new values anytime.

```
int age = 25;
print(age);    // 25
age = 26;      //  Allowed! The value changes.
print(age);    // 26
```

Constants (Cannot Change): `final` vs. `const`

Keyword	Can Change?	When Set?	Example
var	 Yes	At runtime	<code>var score = 0;</code>
final	 No	At runtime	<code>final time = DateTime.now();</code>
const	 No	At compile time	<code>const PI = 3.14;</code>

Code Examples

```
// var (Can Change)
var ageVar = 25;
ageVar = 26; //  Works!

// final (Cannot Change, Set at Runtime)
final ageFinal = 25;
// ageFinal = 26; //  ERROR!
final timeFinal = DateTime.now(); //  Works (reads live clock when app runs)

// const (Cannot Change, Set at Compile Time)
const PI = 3.14159;
```

```
// PI = 3.14; // ❌ ERROR!  
// const timeConst = DateTime.now(); // ❌ ERROR! (Computer can't read live clock before app starts)
```

🎯 How to Choose

- Use **final** when the value is figured out while the app runs:
 - `final currentTime = DateTime.now();` (Live clock time)
 - `final userData = fetchFromAPI();` (Data from the internet)
- Use **const** when the value is a fixed fact known beforehand:
 - `const PI = 3.14159;` (Math constant)
 - `const APP_NAME = "MyApp";` (App name)

💡 Simple Memory Tricks

- `var` = Can change freely anytime. ✅
- `final` = Locked once **while the app is running**. 🕒
- `const` = Locked once **before the app even starts**. ⚡